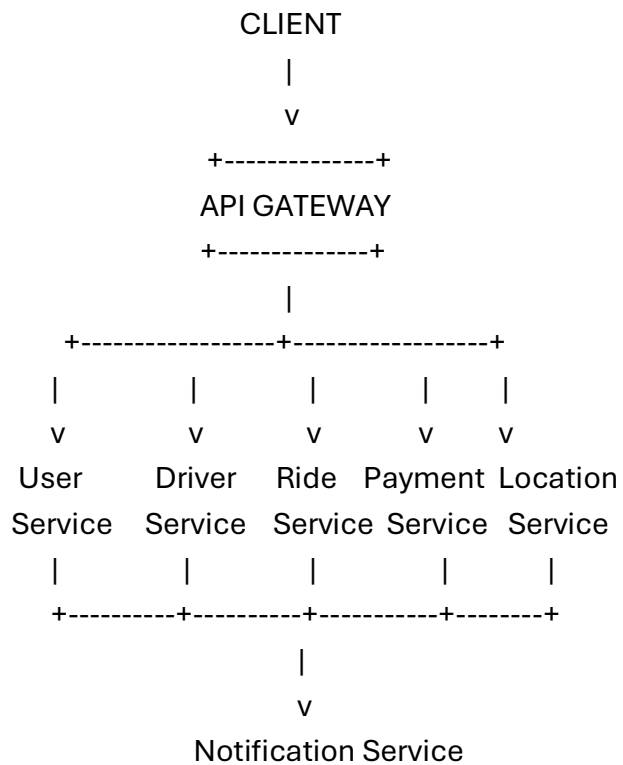


EPIC 09 — Task 02: Microservices Architecture

API Gateway

- User Service
- Driver Service
- Ride Service
- Payment Service
- Location Service
- Notification Service

1. Overall Architecture



2. API Gateway

Responsibility

The API Gateway is the single-entry point for clients.

It handles:

- Routing requests to microservices
- Authentication and authorization
- Request validation
- Rate limiting
- Load balancing
- Centralized logging
- Correlation ID propagation

Dependencies

API Gateway

└─ User Service

└─ Driver Service

└─ Ride Service

└─ Payment Service

└─ Location Service

└─ Notification Service

3. User Service

Responsibility

Manages customer information.

It handles:

- User registration
- Login/authentication
- User profile

- User preferences
- Ride history reference

Dependencies

User Service

└─ User Database

It should own its own user data.

4. Driver Service

Responsibility

Manages driver information and availability.

It handles:

- Driver registration
- Driver profile
- Driver verification
- Online/offline status
- Driver availability
- Driver ride acceptance

Dependencies

Driver Service

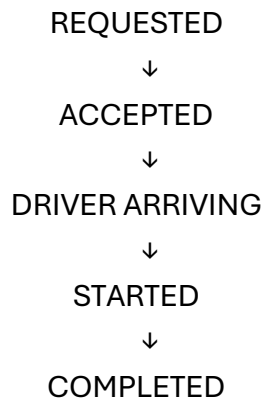
└─ Driver Database

The Driver Service can communicate with the **Location Service** to obtain the driver's current location.

5. Ride Service

Responsibility

Manages the complete ride of lifecycle.

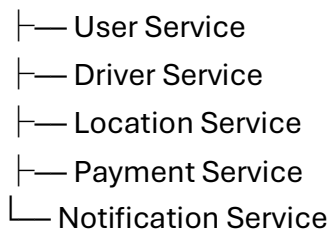


It handles:

- Creating rides
- Finding/assigning drivers
- Ride status
- Pickup and destination
- Fare information
- Ride cancellation
- Ride history

Dependencies

Ride Service



6. Payment Service

Responsibility

Handles all payment operations.

It handles:

- Fare calculation/processing
- Payment initiation
- Payment confirmation
- Payment failure
- Refunds
- Transaction records

Dependencies

Payment Service

└─ Payment Database

It may also communicate with an external payment provider, such as a UPI/payment gateway.

7. Location Service

Responsibility

Manages real-time location information.

It handles:

- Driver GPS location
- User pickup location
- Destination
- Distance calculation
- ETA

- Nearby-driver lookup
- Location updates

Dependencies

Location Service

└─ Location/Geo Database

For high-frequency location data, a cache such as Redis can also be used.

8. Notification Service

Responsibility

Sends notifications to users and drivers.

Examples:

Ride requested

Driver accepted

Driver arriving

Ride started

Ride completed

Payment successful

Ride cancelled

It can support:

- Push notifications
- SMS
- Email

Dependencies

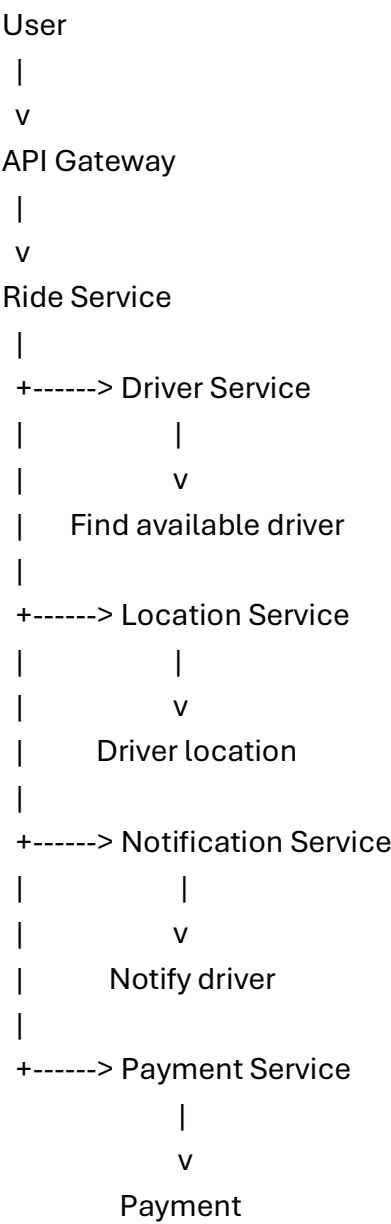
Notification Service

└─ Message Broker

└─ Notification Provider

Service Communication

A typical ride flow would be:



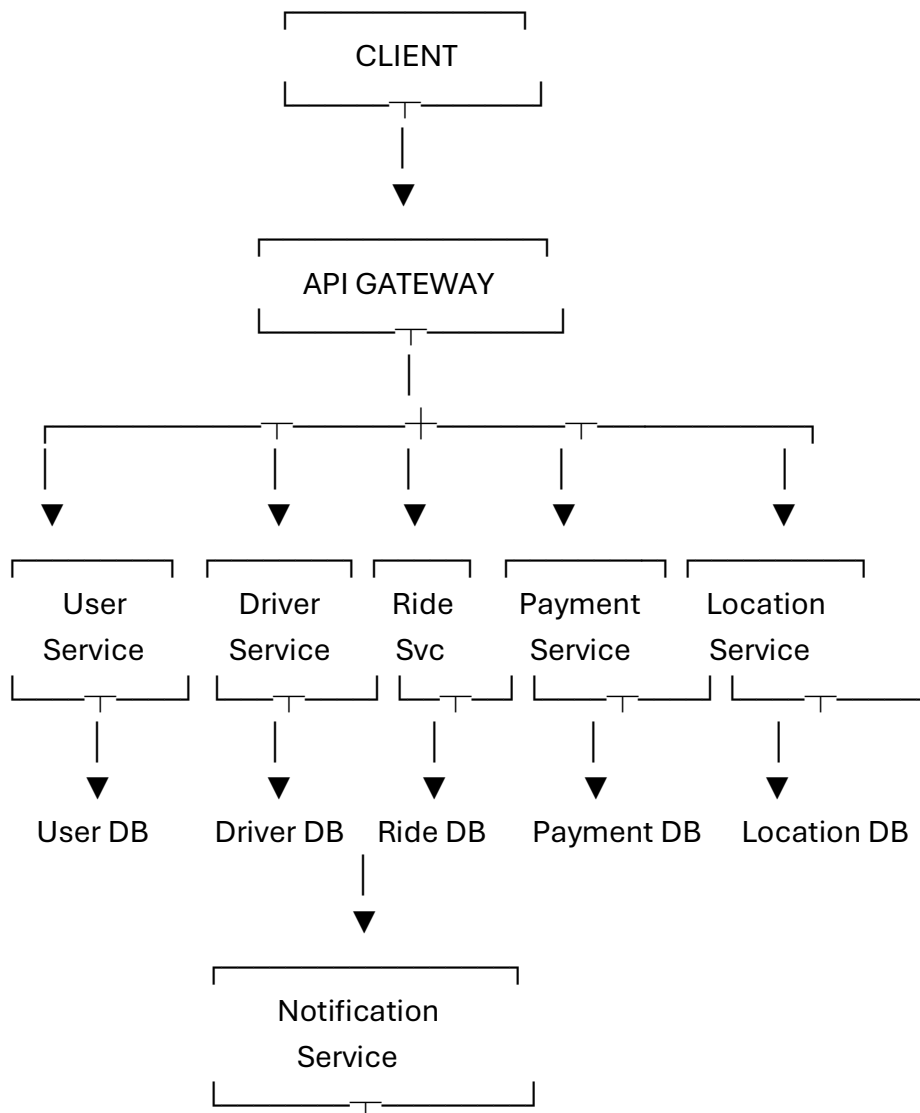
Recommended communication

Communication

Method

Client → Gateway	REST/HTTPS
Gateway → Services	REST/HTTP
Ride → User	REST
Ride → Driver	REST/events
Ride → Location	REST
Ride → Payment	REST/events
Services → Notification	Events/message broker
Location updates	REST/WebSocket/events

Final Architecture





Push / SMS / Email